

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from tensorflow.keras.layers import *
from tensorflow.keras.models import Model

print("Imported Successfully")
```

Imported Successfully

```
images = []
masks = []

image_path = "data/segmentation/images"
mask_path = "data/segmentation/masks"

image_files = sorted([f for f in os.listdir(image_path) if not f.startswith('.')])
mask_files = sorted([f for f in os.listdir(mask_path) if not f.startswith('.')])

if len(image_files) != len(mask_files):
    print(f" Warning: Mismatch in counts! Images: {len(image_files)}")

for img_file, mask_file in zip(image_files, mask_files):

    full_img_path = os.path.join(image_path, img_file)
    full_mask_path = os.path.join(mask_path, mask_file)

    img = cv2.imread(full_img_path)
    mask = cv2.imread(full_mask_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print(f"Skipping: Could not load image file -> {img_file}")
        continue
    if mask is None:
        print(f"Skipping: Could not load mask file -> {mask_file}")
        continue

    img = cv2.resize(img, (128,128))
    mask = cv2.resize(mask, (128,128))

    images.append(img)
    masks.append(mask)

X = np.array(images) / 255.0
y = np.array(masks) / 255.0

y = np.expand_dims(y, axis=-1)

print("Images shape (X):", X.shape)
print("Masks shape (y):", y.shape)
```

```
Images shape (X): (20, 128, 128, 3)
Masks shape (y): (20, 128, 128, 1)
```

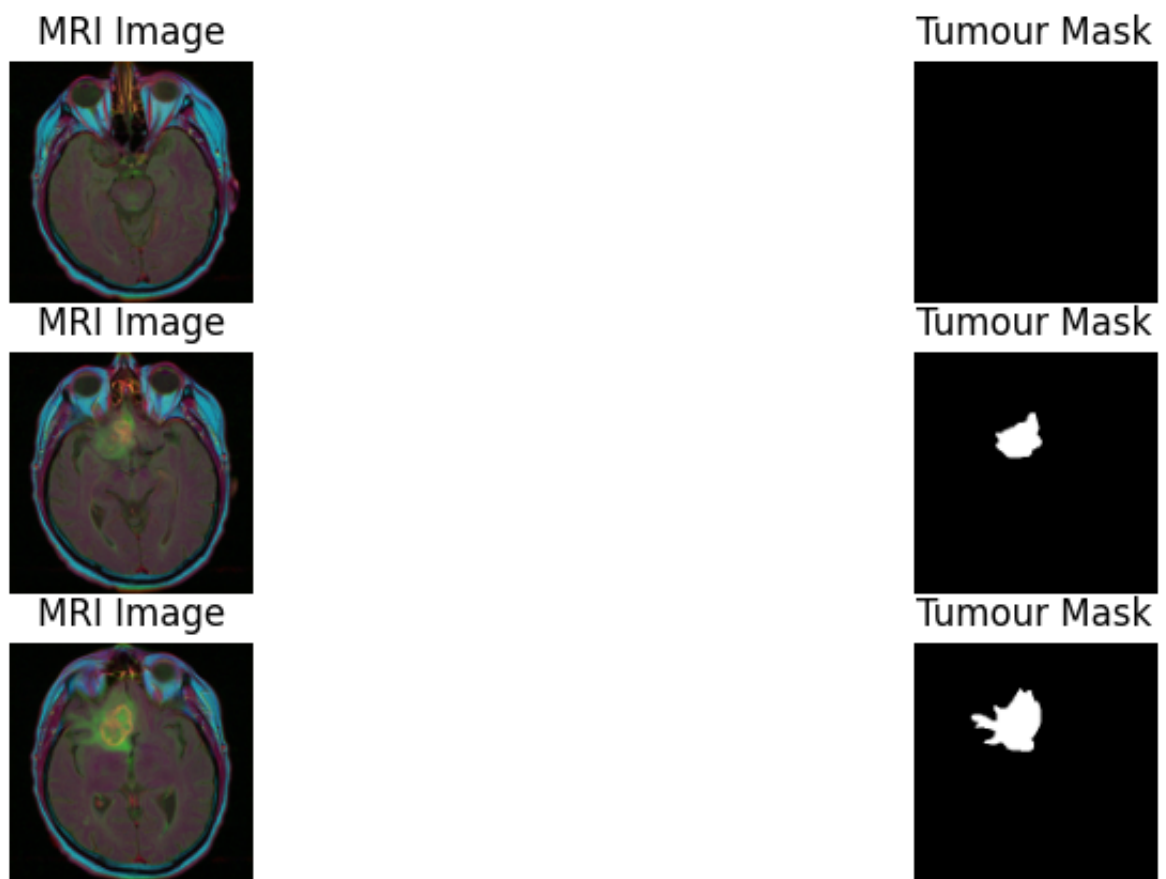
```
plt.figure(figsize=(10,5))

for i in range(3):

    plt.subplot(3,2,2*i+1)
    plt.imshow(X[i])
    plt.title("MRI Image")
    plt.axis("off")

    plt.subplot(3,2,2*i+2)
    plt.imshow(y[i].squeeze(), cmap='gray')
    plt.title("Tumour Mask")
    plt.axis("off")

plt.show()
```



```
inputs = Input((128,128,3))

# Encoder
c1 = Conv2D(16, (3,3), activation='relu', padding='same')(inputs)
p1 = MaxPooling2D((2,2))(c1)

c2 = Conv2D(32, (3,3), activation='relu', padding='same')(p1)
```

```

p2 = MaxPooling2D((2,2))(c2)

# Bottleneck
b1 = Conv2D(64, (3,3), activation='relu', padding='same')(p2)

# Decoder
u1 = UpSampling2D((2,2))(b1)
c3 = Conv2D(32, (3,3), activation='relu', padding='same')(u1)

u2 = UpSampling2D((2,2))(c3)
c4 = Conv2D(16, (3,3), activation='relu', padding='same')(u2)

outputs = Conv2D(1, (1,1), activation='sigmoid')(c4)

model = Model(inputs, outputs)

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "functional"

Layer (type)	Output Shape	F
input_layer (InputLayer)	(None, 128, 128, 3)	
conv2d (Conv2D)	(None, 128, 128, 16)	
max_pooling2d (MaxPooling2D)	(None, 64, 64, 16)	
conv2d_1 (Conv2D)	(None, 64, 64, 32)	
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 32)	
conv2d_2 (Conv2D)	(None, 32, 32, 64)	
up_sampling2d (UpSampling2D)	(None, 64, 64, 64)	
conv2d_3 (Conv2D)	(None, 64, 64, 32)	
up_sampling2d_1 (UpSampling2D)	(None, 128, 128, 32)	
conv2d_4 (Conv2D)	(None, 128, 128, 16)	
conv2d_5 (Conv2D)	(None, 128, 128, 1)	

Total params: 46,689 (182.38 KB)
Trainable params: 46,689 (182.38 KB)

Non-trainable params: 0 (0.00 B)

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_test, y_test),
    epochs=5
)
```

```
Epoch 1/5
1/1 _____ 4s 4s/step - accuracy: 0.3636 - loss: 0.694
Epoch 2/5
1/1 _____ 1s 971ms/step - accuracy: 0.9768 - loss: 0.
Epoch 3/5
1/1 _____ 1s 968ms/step - accuracy: 0.9770 - loss: 0.
Epoch 4/5
1/1 _____ 1s 1s/step - accuracy: 0.9770 - loss: 0.621
Epoch 5/5
1/1 _____ 2s 2s/step - accuracy: 0.9770 - loss: 0.579
```

```
prediction = model.predict(
    np.expand_dims(X_test[0], axis=0)
)[0]
```

```
1/1 _____ 0s 306ms/step
```

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(X_test[2])
plt.title("Original MRI")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(y_test[2].squeeze(), cmap='gray')
plt.title("True Mask")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(prediction.squeeze(), cmap='gray')
plt.title("Predicted Mask")
plt.axis("off")

plt.show()
```



Start coding or [generate](#) with AI.

